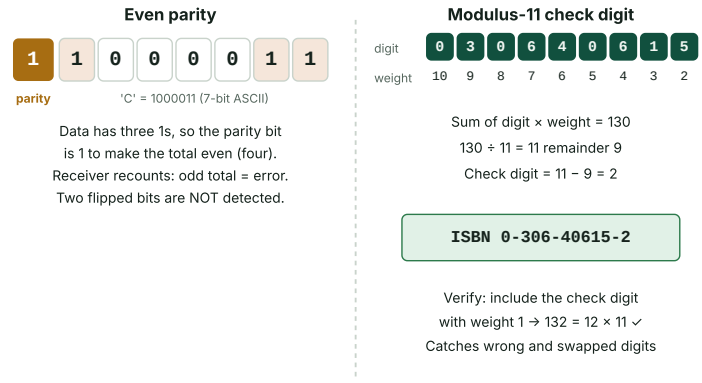
# Character sets and error detection

FD04
FACT FILE

### Specification: students should be able to

- Explain what a character set is and why an agreed standard is needed for text to be exchanged between computers
- Demonstrate how ASCII, extended ASCII and Unicode are used to represent characters
- Compare character sets in terms of the number of characters represented and the storage required
- Describe how odd and even parity bits are used to detect transmission errors, and their limitations
- Explain the purpose of a check digit and how it differs from a parity bit

**Key words:** character set, character code, control character, ASCII, extended ASCII, code page, Unicode, code point, UTF-8, parity bit, even parity, odd parity, check digit, Modulus 11, error detection, transposition error, checksum, echo check




An even parity bit added to the ASCII code for C, and a modulus-11 check digit calculation for an ISBN-10

## Text as numbers: an overview

Everything a computer stores is binary, so text has to be turned into numbers before it can be saved, processed or sent. A character set is the agreed list of characters a system can represent, with a unique binary code for each one. Because the same code must mean the same character on every machine, standards are essential: a message typed on a phone in Omagh must appear identically on a laptop in Lisbon. The oldest widely used standard is ASCII, a 7-bit code with 128 characters. Extended ASCII uses the eighth bit to give 256 characters, and Unicode goes much further, giving a code point to characters from almost every writing system plus symbols and emoji. Because data can also be corrupted as it travels or as it is typed in, systems add extra information to help spot mistakes. A parity bit added to each byte can reveal that a bit has flipped during transmission, and a check digit added to a code, such as one calculated with the Modulus 11 method, can reveal that a number has been keyed in wrongly. This lesson covers both ideas: how characters are represented, and how simple redundancy is used to detect errors.

## How ASCII is organised

ASCII, the American Standard Code for Information Interchange, uses 7 bits, so it has 2 to the power 7, or 128, codes numbered 0 to 127. Codes 0 to 31 are control characters that do not print, such as tab, line feed and carriage return, and code 127 is delete. Code 32 is the space, codes 48 to 57 are the digits, 65 to 90 are the capital letters and 97 to 122 are the lower-case letters. Because the codes are allocated in order, you can work out an unknown code from a known one: if 'A' is 65 then 'H' is 72, and each lower-case letter is exactly 32 more than its capital, which in binary means only one bit is different. The ordering also lets programs sort text by comparing codes, although it means capitals sort before all lower-case letters. It is important to distinguish a digit character from a number: the character '5' is stored as code 53, so arithmetic cannot be done on it until it is converted. A plain text file stores one byte per ASCII character, so a five-letter word saved on its own occupies five bytes, while a word-processed file is larger because formatting is saved too.

### Extended ASCII and Unicode

ASCII has no accented letters, so it cannot store an Irish fada, a French cedilla or a German umlaut, and it has nothing for non-Latin scripts. Extended ASCII uses all eight bits of the byte, giving 256 codes. The first 128 are identical to ASCII with a leading zero added, and codes 128 to 255 hold extra characters. The problem is that there is no single extended ASCII: different code pages give those upper codes different meanings, so a file created with one code page can show the wrong symbols when opened with another. Unicode solves this by giving every character one agreed code point that is the same worldwide. It has room for more than 1.1 million code points, and its first 128 match ASCII so existing ASCII text remains valid. Unicode is stored using an encoding. UTF-8 is the most common on the web: it uses one byte for ASCII characters and two, three or four bytes for others. UTF-16 uses two or four bytes and UTF-32 always uses four. Unicode is now built into all major operating systems, programming languages, browsers and email systems, and new characters such as emoji are added regularly.

### Comparing storage requirements

The choice of character set affects both what can be represented and how much storage and bandwidth text needs. ASCII and extended ASCII always use one byte per character, so a 2,000-character document takes 2,000 bytes. In UTF-8, an English document of the same length is also 2,000 bytes because every ASCII character still needs just one byte, which is why UTF-8 is so popular. A document full of accented letters grows, since each of those needs two bytes; for example, the 12-character name 'Seán Ó Néill' needs 15 bytes in UTF-8 but 12 in a Western European code page. Chinese characters typically need three bytes each in UTF-8. UTF-16 doubles the size of English text and UTF-32 quadruples it, but fixed-width encodings make it quicker to jump to the nth character. A file containing thousands of different Chinese characters must therefore use Unicode, because 256 codes cannot represent them, and the trade-off is a larger file. When answering, link the number of bits to the number of characters using 2 to the power n, and link the extra bytes to larger files and longer transmission times.

### Parity checking

When data is sent along a cable or over a wireless link, electrical interference can flip a bit from 0 to 1 or 1 to 0. A parity bit is a simple way of detecting this. One extra bit is added to each group of data bits, commonly turning a 7-bit ASCII code into a byte. With even parity, the parity bit is chosen so that the byte, parity bit included, holds an even count of 1s; with odd parity that count is odd. Both ends must agree beforehand which system is used. The receiver counts the 1s in every byte it receives; if the count does not match the agreed parity, it knows the byte has been corrupted and asks for it to be sent again. Parity has clear limitations. It cannot tell which bit is wrong, so it cannot correct the error itself. If two bits, or any even number of bits, are flipped in the same byte, the count of 1s stays even or odd and the error passes unnoticed. It also adds overhead, since one bit in every eight carries no data. More robust methods such as checksums and cyclic redundancy checks are used for larger blocks of data.

### Check digits compared with parity

Parity and check digits both add redundant information so that errors can be spotted, but they protect data at different stages. A parity bit is attached to every byte that is transmitted, and it detects bits flipped by interference on the way. A check digit is attached to a code number, such as a book number, a barcode or a membership number, and it detects mistakes made when that number is typed in or scanned. The check digit is calculated from the other digits; whenever the code is entered, the system repeats the calculation and rejects the code if its answer does not match the digit supplied. The Modulus 11 method, used for old ten-digit ISBNs, gives each digit a different weighting, so it catches every single wrong digit and every swap of two digits, known as a transposition error. Its full method, including the special case where X stands for 10, is covered in FD07. Neither technique can correct an error, and a valid check digit only shows that a code is well formed, not that it belongs to the right person or item.

## T Key terms from the mark schemes

**Character set**
An agreed list of characters that a computer system can represent, in which each character is given a unique binary code.
`Credited: A defined list of characters recognised by the hardware and software`

**ASCII**
American Standard Code for Information Interchange: a 7-bit character set of 128 codes covering English letters, digits, punctuation and control characters.

**Extended ASCII**
An 8-bit extension of ASCII giving 256 codes; codes 0–127 match ASCII and codes 128–255 hold accented letters and symbols, which vary between code pages.

**UTF-8**
A variable-length Unicode encoding that uses one byte for ASCII characters and up to four bytes for other characters.

**Parity bit**
An extra bit added to a group of bits so that the total number of 1s is even (even parity) or odd (odd parity), allowing transmission errors to be detected.

**Modulus 11**
A check digit method in which each digit is multiplied by a weighting, the products are added, and the check digit is found from the remainder after dividing by 11 (0 gives 0, 1 gives X, otherwise 11 minus the remainder).
Credited: Mod 11

**Unicode**
A character standard that gives a unique code point to characters from almost all of the world's writing systems, plus symbols and emoji; its first 128 codes match ASCII.

**Control character**
A non-printing character code, such as carriage return, line feed or tab, used to control how text is processed or displayed.

**Check digit**
An extra digit calculated from the other digits of a code number and added to the end, so the computer can detect entry or reading errors by recalculating it.

**Transposition error**
An input error in which two characters are entered in the wrong order, such as 47019 instead of 40719.

## C C# example

Characters as numbers in C#

```
char letter = 'A';
int code = letter;                      // implicit conversion: 65
Console.WriteLine(code);

char lower = (char)(code + 32);         // 97 is 'a'
Console.WriteLine(lower);

string bits = Convert.ToString(code, 2).PadLeft(8, '0');
Console.WriteLine(bits);                // 01000001

Console.WriteLine("7" + "1");           // text joins: 71
Console.WriteLine('7' - '0');           // 55 - 48 = 7
```

### Exam focus

Expect short State and Describe questions on character sets and parity, plus short calculations. When working with ASCII codes, use the ordering of the table to find unknown codes and show the addition or subtraction you used, for example 'K' is 75 so 'N' is 78. For parity, always say whether you are using odd or even, count the 1s explicitly, state the parity bit and give the complete byte. Limitations must be specific: say that two flipped bits leave the parity unchanged, not just that parity is unreliable, and that the receiver cannot tell which bit is wrong. When comparing ASCII and Unicode, quote the number of characters (128, 256, over a million) and give a consequence, such as more languages but more bytes per character. A common error is to claim that UTF-8 always uses two bytes; it uses one to four. If you are asked to compare parity with a check digit, link parity to transmission errors and check digits to data-entry errors. Full Modulus 11 calculations are examined with validation, so practise them using FD07, and do not confuse a check digit, which validates input, with verification by double entry.

## ? Check yourself

1. State what is meant by a character set. [2]
2. The ASCII code for 'K' is 75. State the ASCII code for 'N' and the ASCII code for 'k'. [2]
3. A pupil stores the digit '5' in a string variable and the number 5 in an integer variable. Explain why these are stored differently. [2]

**Answers:** 1. An agreed set of characters that a computer system can recognise and represent (1) / each character has its own unique binary code/number (1). 2. 'N' = 78 (1) / 'k' = 107 (1). 3. The character '5' is stored as its character code (53 in ASCII/Unicode) (1) / the integer 5 is stored as the binary value of 5 (00000101), so arithmetic can only be done directly on the integer (1).